

Abstraction Architecture for AI Coding Agent Harnesses

First Author¹, Second Author², Third Author², Fourth Author¹ and Fifth Author¹

¹Pragnition Labs, ^{*}Equal Contribution, ¹Affiliation 1, ²Affiliation 2

We present a conceptual framework for reasoning about the abstraction spectrum between human natural language and executable code, and its implications for the design of AI coding agent harnesses. Drawing on Dijkstra’s critique of natural language programming, the Curry-Howard-Martin-Löf correspondence, Kolmogorov complexity theory, and the refinement calculus, we define the *abstraction gap* $\mathcal{G}(S, P) = K(P \mid S)$ as the conditional Kolmogorov complexity between a specification S and its implementation P . Applying standard results from algorithmic information theory, we show that this gap is bounded, monotonically decreasing under refinement, and converges to zero precisely when S is informationally equivalent to P , formalising Gonzalez’s (2026) thesis that “a sufficiently detailed spec is code.” We apply this framework to multi-agent and single-agent coding systems following the taxonomy of Kim et al. (2025), analysing how coordination topology (centralised, decentralised, independent, hybrid) modulates the effective abstraction gap. We place the positions of 16 major thinkers across computer science, HCI, and cognitive science on the formalised spectrum using a two-dimensional specificity-compression metric, and motivate six design principles for AI coding agent harnesses

1. Introduction

Recent AI coding agents such as Claude Code¹, OpenCode², and OpenAI Codex³ have demonstrated remarkable programming capabilities. These tools pair frontier large language models (LLMs) with tool-use interfaces (file systems, shell execution, web search, code intelligence) to autonomously read, write, and debug code in iterative agentic loops.

Further, these agents have given rise to *multi-agent* architectures (Kim et al., 2025) where multiple LLM-backed agents coordinate to solve complex software engineering tasks. OpenAI’s Symphony spawns autonomous agents for issue tickets; GitHub Copilot’s coding agent operates asynchronously; Cursor and Windsurf embed agent-mode into IDEs.

Yet a fundamental question remains unresolved: *how much abstraction should exist between human intent (expressed in natural language) and machine execution (expressed in code)?* This question, first posed sharply by Dijkstra in EWD 667 (Dijkstra, 1978), has acquired new urgency as AI agents promise to bridge the gap automatically.

Gonzalez (2026) recently argued that “a sufficiently detailed spec is code”: that as specifications become precise enough to be unambiguous, they necessarily converge on code itself. This thesis, if true, has profound implications for harness design: it suggests that the *interface* between human and AI agent is not a trivial engineering choice but a fundamental design parameter with information-theoretic constraints.

In this paper, we provide a formal mathematical foundation for this debate. This is a *theory and position paper*: we develop formal tools and apply them to derive design principles, but we do not conduct new experiments. Empirical calibration (Section 9) draws on published benchmarks

¹<https://claude.com/product/claude-code>

²<https://opencode.ai/>

³<https://openai.com/codex/>

and studies. We note that this is a rapidly evolving area where practitioner discourse (blog posts, industry reports, and preprints) frequently precedes peer-reviewed publication; we draw on such sources where they provide the best available evidence, and flag them accordingly. Our contributions are:

1. A definition of the **abstraction gap** as $K(P \mid S)$, applying standard results from algorithmic information theory to the specification-code relationship (§3). The novel application is Theorem 3.3 (Spec-Code Convergence), which formalises Gonzalez’s thesis for incompressible programs.
2. A **refinement lattice** synthesis connecting Dijkstra’s weakest precondition calculus to the Curry-Howard correspondence and the information-theoretic gap, placing specifications and code in a unified formal framework (§4).
3. An analysis of **single-agent and multi-agent** coding systems through the lens of the abstraction gap, drawing on the empirical findings of Kim et al. (2025) (§5).
4. A **two-dimensional specificity-compression metric** organising the positions of 16 major thinkers on the formalised spectrum as a pedagogical and analytical device (§6).
5. Six **design principles** for AI coding agent harnesses motivated by the formal framework and consistent with current engineering practice (§11).

2. Preliminaries and Definitions

2.1. Agent Systems

Following Kim et al. (2025), we define agent systems as follows.

Definition 2.1 (Agent). An agent a is a tuple $a = (\Phi, \mathfrak{A}, \mathcal{T}, \pi)$, where:

- Φ is the reasoning policy (typically an LLM with parameters θ),
- $\mathfrak{A}$ is a finite set of available actions (tools: file read/write, shell execution, web search, etc.),
- $\mathcal{T} = \{t_1, \dots, t_k\}$ is a set of tools providing environmental interaction,
- $\pi : O^* \rightarrow \Delta(\mathfrak{A})$ is the agent’s policy, mapping observation histories to distributions over actions.

Definition 2.2 (Single-Agent System (SAS)). A single-agent system is a tuple $S_{SAS} = (\{a\}, \mathcal{E}, \emptyset)$ where $|A| = 1$, $\mathcal{E}$ is the environment (codebase, terminal, file system), and there is no inter-agent communication. The agent operates in an agentic loop: observe $\rightarrow$ reason $\rightarrow$ act $\rightarrow$ verify $\rightarrow$ repeat, consuming a token budget B .

Definition 2.3 (Multi-Agent System (MAS)). A multi-agent system is a tuple $S_{MAS} = (A, \mathcal{E}, C, \Omega)$ where $|A| > 1$, C is a communication topology (a directed graph over agents), and Ω is an orchestration policy governing task assignment, aggregation, and termination. Following Kim et al. (2025), we distinguish four canonical topologies:

- **Independent:** Agents operate in isolation; outputs aggregate at endpoint. $C = \emptyset$.
- **Centralised:** A single orchestrator coordinates r rounds across n sub-agents. C is a star graph.
- **Decentralised:** Peer-to-peer communication without hierarchy. C is a (possibly complete) undirected graph.
- **Hybrid:** Orchestrator control with selective peer links. C combines star and peer edges.

2.2. Coordination Metrics

Kim et al. (2025) define four empirical coordination metrics:

Definition 2.4 (Coordination Efficiency). $E_c = S / (T_{\text{MAS}} / T_{\text{SAS}})$, where S is the success rate and T is the turn count. This measures effectiveness per unit interaction cost, normalised against SAS baseline.

Definition 2.5 (Coordination Overhead). $O = (T_{\text{MAS}} - T_{\text{SAS}}) / T_{\text{SAS}} \times 100\%$, quantifying extra turns consumed by multi-agent interaction.

Definition 2.6 (Error Amplification). $A_e = E_{\text{MAS}} / E_{\text{SAS}}$, the ratio of MAS to SAS error rates. Values > 1 indicate error propagation; < 1 indicates error correction.

Definition 2.7 (Redundancy Rate). $R = \text{mean}(\cos(e_i, e_j))$ for agent output embeddings e_i, e_j , capturing output overlap.

Kim et al. (2025) find that independent agents amplify errors 17.2× while centralised coordination reduces this to 4.4×, and that capability saturation occurs at ~45% single-agent performance ($\beta = -0.408$, $p < 0.001$).

2.3. The Specification-Code Pair

Definition 2.8 (Specification). A specification S is a predicate transformer $S : \text{Pred}(\Sigma) \rightarrow \text{Pred}(\Sigma)$ over a state space Σ , in the sense of Dijkstra’s weakest precondition calculus (Dijkstra, 1976). Equivalently, S may be viewed as: a logical proposition (Curry-Howard), a type in a dependent type system (Martin-Löf, 1979), or an object in a Cartesian closed category (Lambek and Scott, 1986).

Definition 2.9 (Program). A program P is a specification that is additionally deterministic (for each initial state, at most one final state) and feasible (it does not satisfy contradictory postconditions). Equivalently, P is a term of a given type, a proof of a given proposition, or a morphism from the terminal object in a CCC.

Definition 2.10 (Implementation Relation). Program P implements specification S (written $P \models S$) iff $S \sqsubseteq P$ in the refinement ordering: $\forall Q \in \text{Pred}(\Sigma). \text{wp}(S, Q) \subseteq \text{wp}(P, Q)$.

3. The Abstraction Gap: Information-Theoretic Foundations

3.1. Kolmogorov Complexity of Specifications and Code

We define the abstraction gap using standard results from algorithmic information theory (Chaitin, 1966; Kolmogorov, 1965; Li and Vitányi, 2019). We do not claim novelty for the information-theoretic machinery itself; our contribution is its systematic application to the specification-code relationship in the context of AI coding agents, and the derived consequences for harness design.

Definition 3.1 (Abstraction Gap). For a specification S and program P (both encoded as finite binary strings), the abstraction gap is:

$$\mathcal{G}(S, P) \triangleq K(P \mid S) \tag{1}$$

where $K(P \mid S) = \min\{|p| : U(p, S) = P\}$ is the conditional Kolmogorov complexity, i.e., the length of the shortest program that, given S as auxiliary input, produces P on a universal Turing machine U .

Theorem 3.1 (Properties of the Abstraction Gap). The abstraction gap satisfies:

1. **Minimality:** $\mathcal{G}(S, P) = O(1)$ iff S algorithmically determines P .
2. **Maximality:** $\mathcal{G}(S, P) = K(P) + O(1)$ iff S provides no information about P .

3. **Monotonicity:** If $S \sqsubseteq S'$ in the refinement ordering, then $\mathcal{G}(S', P) \leq \mathcal{G}(S, P) + O(\log n)$ where $n = \max(|S|, |S'|, |P|)$. If the refinement is additionally constructive (i.e., there exists a computable function f of bounded description length with $f(S') = S$), the bound tightens to $\mathcal{G}(S', P) \leq \mathcal{G}(S, P) + O(1)$.
4. **Additivity:** For intermediate representation S' , $\mathcal{G}(S, P) \leq \mathcal{G}(S, S') + \mathcal{G}(S', P) + O(\log n)$ where $n = \max(|S|, |P|)$.

Proof sketch. (a) follows from the definition: if $K(P | S) = O(1)$, a constant-length program suffices to produce P from S . (b) follows from $K(P | S) \leq K(P) + O(1)$ (ignoring S is always an option) with equality when S is independent of P . (c) follows from the chain rule: $K(P | S') \leq K(S | S') + K(P | S) + O(\log n)$; under standard refinement the $K(S | S')$ term contributes $O(\log n)$, and under constructive refinement it is $O(1)$. (d) follows from the chain rule for Kolmogorov complexity (Kolmogorov, 1965): $K(x, y) = K(x) + K(y | x, K(x)) + O(\log n)$. $\square$

3.2. The Normalised Information Distance

Following Li et al. (2004), we define the canonical metric on the abstraction spectrum:

Definition 3.2 (Abstraction Distance). *The abstraction distance between specification S and program P is the normalised information distance:*

$$d(S, P) = \frac{\max(K(S | P), K(P | S))}{\max(K(S), K(P))} \quad (2)$$

Theorem 3.2 (Universality, Li et al., 2004). *d is a metric (up to $O(\log n/n)$ terms) and is universal: for every computable normalised distance $D : \{0, 1\}^* \times \{0, 1\}^* \rightarrow [0, 1]$, we have $d(x, y) \leq D(x, y) + O(1/n)$.*

This establishes $d(S, P) \in [0, 1]$ as the canonical metric for the abstraction spectrum: $d \approx 0$ means the specification and code are informationally equivalent; $d \approx 1$ means they share no algorithmic content.

3.3. Convergence Theorem

Theorem 3.3 (Spec-Code Convergence). *For an algorithmically random program P (i.e., $K(P) \geq |P| - O(1)$), any specification S such that $P \models S$ and $\mathcal{G}(S, P) = O(1)$ satisfies:*

$$|S| \geq |P| - O(\log |P|) \quad (3)$$

That is, a specification that uniquely determines an incompressible program must be at least as long as the program itself, up to logarithmic terms.

Proof. By Chaitin’s incompleteness theorem (Chaitin, 1966), an N -bit formal system can determine at most $N + O(1)$ bits of an algorithmically random string. Since $K(P) \geq |P| - O(1)$ and $\mathcal{G}(S, P) = O(1)$ implies $K(P) \leq K(S) + O(1)$, we have $|S| \geq K(S) \geq K(P) - O(1) \geq |P| - O(1)$. $\square$

This is the information-theoretic proof that “a sufficiently detailed spec is code” (Gonzalez, 2026): for complex (incompressible) programs, the specification must contain essentially all the information of the program.

3.4. The Shannon Channel Model

We model spec-to-code translation as a noisy channel (Shannon, 1948):

Definition 3.3 (Spec-to-Code Channel). *Let $\mathcal{P}$ be the space of programs and $\mathcal{S}$ the space of specifications. The spec-to-code channel is characterised by transition probabilities $p(P | S)$, with capacity:*

$$C = \max_{p(S)} I(S; P) = \max_{p(S)} [H(P) - H(P | S)] \quad (4)$$

For an LLM-based agent translating natural language to code, the channel noise includes ambiguity, hallucination, and implementation choice. Hindle et al. (2012) measured Java source code cross-entropy at 3–4 bits/token, while Shannon (1951) measured English at 0.6–1.3 bits/character (note these are different units; normalisation requires a token-to-character mapping that varies by language). The entropy gap between these representations provides a rough bound on the information that must be supplied by the agent’s prior knowledge (training data). Deriving a tight capacity bound for the spec-to-code channel, accounting for the LLM’s learned prior, is an important direction for future work.

Relationship between the K-based and Shannon frameworks. The Kolmogorov complexity framework (Sections 3–4) and the Shannon channel model serve complementary roles. The K-based abstraction gap $\mathcal{G}(S, P) = K(P | S)$ is the *conceptual foundation*: it is distribution-free, connects to the refinement calculus, and yields the convergence theorem (Theorem 3.3). However, K is uncomputable. The Shannon channel model provides the *operational counterpart*: conditional entropy $H(P | S)$ is estimable from data, the chain rule holds with exact equality (no $O(\log n)$ terms), and the probabilistic framework naturally fits LLM-mediated translation. The key results of this paper (monotonicity under refinement, convergence, agent decomposition) have direct Shannon analogues. We view the K-based framework as providing the theoretical ceiling and the Shannon model as providing the measurable, falsifiable floor; a complete theory of harness design will require both.

4. The Refinement Lattice

4.1. Specifications as a Complete Lattice

Theorem 4.1 (Refinement Lattice, Back, 1980; Morgan, 1994). *The set of specifications over a state space Σ , ordered by refinement $\sqsubseteq$, forms a complete lattice where:*

- $\top$ = **abort**: establishes no postcondition (maximally abstract, any program satisfies it).
- $\perp$ = **magic**: establishes every postcondition (impossible perfect program).
- $S_1 \sqcap S_2$ (meet): demonic choice. $(S_1 \sqcap S_2)(Q) = S_1(Q) \cap S_2(Q)$.
- $S_1 \sqcup S_2$ (join): angelic choice. $(S_1 \sqcup S_2)(Q) = S_1(Q) \cup S_2(Q)$.

A concrete, deterministic, feasible program P sits near the bottom of this lattice. The **abstraction level** of a specification is its position: higher (toward $\top$) means more abstract; lower (toward $\perp$) means more concrete. “A sufficiently detailed spec is code” becomes: *sufficiently many refinement steps reach a deterministic, feasible element of the lattice.*

4.2. The Curry-Howard-Lambek Bridge

The Curry-Howard-Lambek correspondence (Lambek and Scott, 1986; Martin-Löf, 1979; Wadler, 2015) unifies the refinement lattice with type theory and category theory:

Logic	Type Theory	Category Theory
Proposition	Type	Object
Proof	Term (program)	Morphism $1 \rightarrow A$
Implication $A \Rightarrow B$	Function $A \rightarrow B$	Exponential B^A
$\forall x:A. B(x)$	$\Pi(x:A).B(x)$	Right adjoint to pullback
$\exists x:A. B(x)$	$\Sigma(x:A).B(x)$	Left adjoint to pullback
Cut elimination	β -reduction	Composition

Under this correspondence, a *specification* is a proposition/type, and an *implementation* is a proof/term. Martin-Löf’s intuitionistic type theory (Martin-Löf, 1979) establishes a structural isomorphism between these objects: specifications and implementations inhabit a single formal universe. The gap between them is a gap of *role within a shared structure*, not of mathematical kind. This does not entail that specifications and implementations are interchangeable in practice (no working mathematician treats the statement of a theorem as identical to its proof), but it does mean they are related by a formally precise notion of refinement rather than by an informal analogy.

4.3. Galois Connections as Abstraction Layers

Following Cousot and Cousot (1977), each level of the abstraction spectrum corresponds to a Galois connection:

Definition 4.1 (Abstraction Layer). *An abstraction layer between concrete domain $(C, \sqsubseteq_C)$ and abstract domain $(A, \sqsubseteq_A)$ is a Galois connection (α, γ) where $\alpha : C \rightarrow A$ (abstraction) and $\gamma : A \rightarrow C$ (concretisation) satisfy:*

$$\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a) \quad (5)$$

The abstraction spectrum is then a *tower of Galois connections*:

$$C \rightrightarrows A_1 \rightrightarrows A_2 \rightrightarrows \dots \rightrightarrows A_n \quad (6)$$

where C is executable code and A_n approaches natural language. Each layer discards information; the composition $\alpha_n \circ \dots \circ \alpha_1$ maps code to its most abstract description. Information loss is monotonic.

5. Application to Multi-Agent Systems

5.1. The Agent-Mediated Abstraction Gap

When an AI coding agent mediates between specification and code, the abstraction gap decomposes:

Remark (Agent Decomposition). *For an agent a receiving specification S and producing program P , the abstraction gap decomposes via the chain rule (Theorem 3.1d) as:*

$$\mathcal{G}(S, P) \leq \underbrace{\mathcal{G}(S, \hat{S})}_{\text{interpretation gap}} + \underbrace{\mathcal{G}(\hat{S}, P)}_{\text{generation gap}} + O(\log n) \quad (7)$$

where $\hat{S}$ is any intermediate representation (here, the agent’s “understood specification”). This is a direct application of the chain rule to the string $\hat{S}$; the agent-specific insight is the interpretation: the first term measures how much the agent misunderstands the spec, and the second measures how much implementation detail remains after understanding.

In a single-agent system, both gaps are borne by one model. In a multi-agent system, the gaps can be distributed.

5.2. Multi-Agent Gap Modulation

Following the taxonomy of [Kim et al. \(2025\)](#), different coordination topologies modulate the effective abstraction gap differently. Qualitatively, coordination topology affects the gap through two competing mechanisms:

- **Coordination overhead and error propagation** increase the effective gap: inter-agent communication introduces noise, and errors in one agent’s output propagate through the system, adding implementation uncertainty.
- **Output diversity** decreases the effective gap: independent agents explore different regions of the implementation space, providing redundant “guesses” that increase the probability of finding a correct refinement.

A rigorous information-theoretic model connecting the coordination metrics of [Kim et al. \(2025\)](#) (overhead O , error amplification A_e , redundancy R) to conditional Kolmogorov complexity remains an important open problem. The difficulty lies in the dimensional heterogeneity of these metrics: $\mathcal{G}$ is measured in bits, while O , A_e , and R are ratios and similarities with no natural information-theoretic interpretation. We therefore present the empirical findings qualitatively rather than proposing an ad hoc formula.

From the empirical findings of [Kim et al. \(2025\)](#):

Empirical findings by topology ([Kim et al., 2025](#)).

1. **Independent MAS:** Maximises diversity ($R \rightarrow 0$) but amplifies errors ($A_e = 17.2\times$). Effective for parallelisable tasks where individual sub-problems are well-specified.
2. **Centralised MAS:** Minimises error amplification ($A_e = 4.4\times$) through orchestrator review, but introduces coordination overhead ($T \propto n^{1.7}$). Effective when a single orchestrator can decompose the specification.
3. **Decentralised MAS:** Best for web navigation (+9.2%), where situated feedback reduces the interpretation gap. But sequential reasoning degrades 39–70%.
4. **Capability saturation:** Multi-agent gains diminish when single-agent performance exceeds ~45% ($\beta = -0.408$, $p < 0.001$).

The key insight for harness design: *the coordination topology of the agent system is itself a parameter of the abstraction gap*. A harness must select topology based on task structure, not apply a fixed architecture universally.

6. The Specificity-Compression Metric and Thinker Placement

6.1. Two Dimensions of Abstraction

Empirical analysis reveals that abstraction requires *two* independent dimensions. We note that both dimensions defined below are uncomputable in the general case (by Rice’s theorem for σ and the uncomputability of K for κ). They serve as *idealised* measures that can be approximated via computable proxies: function point backfiring ratios for κ (see Appendix C), and bounded model-checking coverage or type-system expressiveness for σ . The numerical values assigned to thinkers in Table 1 are ordinal estimates based on their published positions, not computed quantities.

Definition 6.1 (Specificity-Compression Space). Fix a size bound N and let $\text{Prog}_N = \{P : |P| \leq N\}$ be the finite set of programs of length at most N . The abstraction position of an artifact S relative to program $P \in \text{Prog}_N$ is a point $(\sigma, \kappa) \in [0, 1]^2$ where:

- $\sigma(S) = 1 - \frac{\log |\llbracket S \rrbracket_N|}{\log |\text{Prog}_N|}$ is the specificity (how precisely S constrains the implementation space $\llbracket S \rrbracket_N \subseteq \text{Prog}_N$), where $|\llbracket S \rrbracket_N|$ is the number of programs in Prog_N satisfying S .
- $\kappa(S) = \max\left(0, 1 - \frac{|S|}{K(P)}\right)$ is the compression (how concise S is relative to the program's Kolmogorov complexity). The $\max$ ensures $\kappa \in [0, 1]$; values that would otherwise be negative (as when a full formal proof exceeds the program in length, e.g., seL4's 200k-line Isabelle proof for 8.7k lines of C) are clamped to zero, indicating that the specification provides no compression.

The bound N is a parameter of the metric; in practice, it is determined by the deployment context (e.g., N = maximum project size in bits). As $N \rightarrow \infty$, $\sigma \rightarrow 0$ for any finitely satisfiable specification, reflecting the intuition that no finite spec constrains an infinite program space. We note that even for finite N , computing σ exactly requires deciding which programs in Prog_N satisfy S , which is undecidable in general by Rice's theorem. The metric therefore serves as an idealised measure; the thinker placements in Table 1 are expert ordinal estimates, not approximations of a computable quantity.

TLA+ is both concise ($\kappa \approx 0.9$) and precise ($\sigma \approx 0.8$) (Newcombe et al., 2015). Natural language is concise ($\kappa \approx 0.9$) but vague ($\sigma \approx 0.1$). This explains why TLA+ and English have similar lengths for the same system but radically different implementation reliability.

6.2. Placement of Major Thinkers

We use the two-dimensional framework to organise the positions of 16 major thinkers qualitatively. The placements below reflect our reading of their published positions and are intended as a pedagogical device for mapping the design space, not as approximations of a computable quantity:

Thinker	Advocated Level	$\hat{\sigma}$ (est.)	$\hat{\kappa}$ (est.)	Framework
Martin-Löf	Types = Proofs = Programs	1.0	~ 0	Intuitionistic TT
Dijkstra	Formal mathematical proof	0.9–1.0	0.0–0.2	Weakest precondition
Brady	Dependent types	0.9–0.95	0.1–0.2	Idris 2 / QTT
Hoare	Axiomatic specs $\{P\}S\{Q\}$	0.85–0.9	0.2–0.4	Hoare logic
Lamport	Mathematical spec above code	0.75–0.8	0.8–0.9	TLA+
Meyer	Contracts in code	0.7–0.75	0.15–0.25	Design by Contract
Harel	Visual formalism	0.65–0.7	0.5–0.7	Statecharts
Knuth	Literate programming	0.6–0.65	~ 0	WEB/CWEB
Parnas	Interface specs < code	0.5–0.6	0.6–0.8	Information hiding
Jackson	Concept design	0.45–0.55	0.7–0.8	Alloy
Brooks	Problem-dependent	var.	var.	Essential complexity
Chollet	Abstraction as core capability	0.3–0.7 [†]	0.5–0.8 [†]	ARC / abstraction
Kay	Interactive exploration	0.2–0.5 [†]	0.3–0.6 [†]	Smalltalk / STEPS
Wittgenstein	Language-game dependent	$\ddagger$	$\ddagger$	Phil. Investigations
Suchman	Situated action	$\ddagger$	0.8–0.9	Plans & Sit. Actions
Karpathy	English as PL	0.05–0.15	0.85–0.95	LLM-mediated

Table 1 | Placement of major thinkers on the specificity-compression plane. Values are ordinal estimates based on each thinker’s published positions, not computed quantities. [†]Range reflects the thinker’s position that the appropriate level varies by task or context; we estimate the range their work typically occupies. [‡]These thinkers reject the premise that their position can be mapped onto a fixed (σ, κ) point (see text). Three clusters emerge: the *formal cluster* ($\hat{\sigma} > 0.7$), the *middle ground* ($0.4 < \hat{\sigma} < 0.7$), and the *NL cluster* ($\hat{\sigma} < 0.3$). Subject to the ordinal reliability of these placements, we hypothesise a gap at $0.3 < \hat{\sigma} < 0.45$, the “uncanny valley” of specification.

6.3. The Uncanny Valley of Specification

The placements suggest a hypothesis: the range $\sigma \in [0.3, 0.45]$ is sparsely populated, forming an “uncanny valley” of specification. Specifications in this zone are formal enough to feel constraining but not formal enough to provide mechanical guarantees. This is the zone of semi-formal notations (UML, most design documents) that are widely criticised as providing the worst of both worlds (Jackson, 2021). The formal framework offers an explanation: this zone maximises cognitive cost (the spec requires careful reading) while minimising verification benefit (the spec cannot be mechanically checked). We emphasise that this is a hypothesis derived from ordinal placements in a 16-row table, not a measured finding. A controlled study comparing developer productivity across formalism levels would be needed to test it.

7. Cognitive Factors

The formal framework operates in the context of human cognitive constraints that affect where on the abstraction spectrum a given team can productively operate. We note these briefly without claiming that the cognitive science literature uniformly supports any single point on the spectrum.

The choice of specification formalism interacts with human cognition in at least two ways. First, as Iverson (Iverson, 1980) argued, notation shapes what practitioners find natural to express: formal languages make invariants, concurrency properties, and type constraints convenient to state, while natural language makes intent, rationale, and context convenient to convey. The practical operating point on the (σ, κ) spectrum depends on which properties matter most for a given component. Second, the usability of a specification formalism depends on factors that resist simple ordering: what Green and Petre (Green and Petre, 1996) call *cognitive dimensions* (viscosity, visibility, premature

commitment, hidden dependencies) vary across formalisms in ways that do not align with a single “more formal is better” axis. A highly formal specification that scores poorly on visibility may be less effective than a semi-formal one that makes key properties salient.

These considerations reinforce Design Principle 2 (support multiple formalism levels): the appropriate level of formality depends not only on the information-theoretic properties of the specification but also on the cognitive characteristics of the team and the task.

8. Contrarian Analysis: The Case for Natural Language

Several intellectual traditions challenge the framework’s assumptions in ways that deserve genuine engagement, not merely cataloguing.

The Bitter Lesson (Sutton, 2019). General methods that scale (LLMs trained on all code) may outperform engineered formal methods. Dijkstra’s approach (designing formal languages for formal thought) is the epitome of the approach that scaling predicts will lose. Our framework does not predict whether scaling will obsolete formal methods. If it does, the abstraction gap $\mathcal{G}(S, P)$ will still exist; what will change is the mechanism for traversing it (brute-force learned priors rather than structured refinement). The framework is agnostic on mechanism.

Distributed Cognition (Clark, 2003; Hutchins, 1995). The human + LLM + IDE + tests system constitutes a distributed cognitive system. The system achieves precision even if no single component (including the human’s NL specification) is precise alone. Formally: $\mathcal{G}(S_{\text{NL}}, P)$ may be large, but $\mathcal{G}(S_{\text{NL}} \parallel \Phi_{\text{LLM}} \parallel \mathcal{T}_{\text{tests}}, P)$ can be small, where $\parallel$ denotes composition in the distributed system. This is a genuine strength of the NL approach that the framework can accommodate but does not predict.

Situated Action (Suchman, 1987). Suchman argued that plans are *resources for action*, not *determinants of action*. If this is correct, the refinement lattice (which models specification as a plan that is progressively concretised) may be the wrong metaphor for iterative, conversational development. The situated action view suggests that the specification-code relationship is not a static refinement chain but a dynamic, context-dependent process in which meaning emerges from interaction. Our framework captures the information-theoretic aspect of this relationship but not the situated, improvisational aspect. For AI coding agents that operate through conversation (iterative prompting, evaluation, and revision), Suchman’s critique is particularly sharp: the “specification” is not a document but an evolving dialogue whose informational content is not fixed at any single moment.

Language Games (Wittgenstein, 1953). If meaning is constituted by use rather than by formal structure, then a prompt’s informational content is not a fixed quantity measurable by Kolmogorov complexity; it is a *move* in a language game whose significance depends on the shared practices of the participants. This challenges a presupposition of the entire information-theoretic framework: that specifications have fixed informational content independent of context. We acknowledge this as a genuine limitation. The framework applies most naturally when specifications are written documents with stable content, and less naturally to interactive, conversational development where meaning is negotiated in real time.

Democratisation. 5 billion NL speakers vs. 27 million programmers (185:1 ratio). If NL programming enables even 10% of this population to direct computation, the total value created dwarfs the quality loss on individual programs (Altman, 2024; Litt, 2023).

These arguments are strongest when the specificity requirement is low ($\sigma < 0.3$) and weakest when formal guarantees are needed ($\sigma > 0.7$). The framework accommodates both by treating σ

as a design parameter, not a universal constant. However, we note that Suchman and Wittgenstein would challenge the σ parameter itself as an illegitimate formalisation of something that resists quantification. We do not resolve this tension; we flag it as a foundational question for future work on the epistemology of specification.

9. Empirical Calibration

9.1. Benchmark Evidence

Benchmark	Top Score	Measures	Gap Indicator
HumanEval	>90%	Isolated functions	Saturated
SWE-bench Verified	77.2%	Real GitHub issues	METR: ~50% not mergeable
LiveCodeBench	38.9%	Dynamic real-world	Still challenging
Dafny VeriCoding	82%	Formally verified	Most promising

Table 2 | Benchmark pass rates quantify the abstraction gap at different σ levels. The gap between HumanEval (high σ tasks, short specs) and LiveCodeBench (low σ tasks, complex specs) is >50 percentage points.

The METR study (METR, 2026) finding that ~50% of test-passing PRs would not be merged confirms Brooks’s insight (Brooks, 1987): passing tests (low- σ verification) is necessary but not sufficient; code quality, architectural coherence, and tacit standards (high- σ properties) remain unverified.

9.2. Code Quality Degradation

GitClear (2025) found that AI-assisted codebases show 17% maintainability drops, refactoring declining from 25% to <10% of changed lines, and bug rates rising 12% at 6 months. This is the empirical cost of operating at low σ : the specification (prompt) underdetermines the implementation, and the implementation choices made by the LLM drift from architectural intent.

9.3. The Vericoding Pattern

The most promising empirical result is “vericoding” (VeriCoding, 2025): human writes formal specification → AI generates implementation → formal verifier checks conformance. The 82% success rate on Dafny specifications demonstrates that the $\sigma \approx 0.8$ operating point is practically achievable today, with the formal verifier providing a deterministic quality gate that neutralises LLM hallucination (Kleppmann, 2025).

10. Related Work

Refinement calculus and program specification. The refinement lattice (Section 4) builds directly on Back’s refinement calculus (Back, 1980) and Morgan’s *Programming from Specifications* (Morgan, 1994), which formalised the stepwise refinement of specifications into code. Hoare and He’s Unifying Theories of Programming (UTP) (Hoare and He, 1998) went further, showing that programs are predicates and refinement is logical implication, placing specifications and code in a single semantic universe. Abrial’s B-Method and Event-B (Abrial, 2010) provide the most industrially deployed refinement-based formal method, with verified systems including the Paris Météor driverless line, and

directly instantiate the tower of Galois connections described in Section 4. Our contribution is not to extend these formalisms but to connect them to the information-theoretic measure $\mathcal{G}(S, P) = K(P \mid S)$ and to apply the combined framework to AI agent harness design.

Abstract interpretation. Cousot and Cousot’s abstract interpretation framework (Cousot and Cousot, 1977) formalised abstraction via Galois connections five decades ago. Subsequent work developed the theory into a mature discipline with industrial tool support (Astrée, Polyspace, Infer). We use the Galois connection formalism to model the abstraction spectrum (Section 4), but we do not extend the abstract interpretation framework itself.

Algorithmic information theory and software. The application of Kolmogorov complexity to software is not new. Li and Vitányi (2019) discuss the connection between algorithmic complexity and program equivalence. The normalised information distance (Li et al., 2004) has been applied to software clone detection and plagiarism, though not (to our knowledge) to the specification-code relationship specifically.

Multi-agent coding systems. Kim et al. (2025) provide the empirical foundation for our multi-agent extension, characterising scaling laws across five canonical architectures. Other recent work on multi-agent coding includes ChatDev, MetaGPT, and AgentCoder, which explore role-based agent decomposition for software tasks. Our contribution is to connect these empirical findings to the abstraction gap framework.

Formal methods and AI. Kleppmann (Kleppmann, 2025) predicted that AI would make formal verification mainstream; Congdon (Congdon, 2025) subsequently endorsed and extended this argument with a concrete workflow vision. The vericoding benchmark (VeriCoding, 2025) provides early empirical support. Our framework formalises *why* this convergence is expected: AI reduces the cost of traversing the refinement lattice, while formal verification provides a deterministic oracle at the bottom.

Position in the landscape. This paper is a *synthesis and formalisation*, not a claim of new mathematical results. The information-theoretic machinery is standard (Li and Vitányi, 2019); the refinement calculus is established (Back, 1980; Morgan, 1994); the Curry-Howard correspondence is a theorem (Wadler, 2015). Our contribution is the application of these tools to the specific question of harness design, the two-dimensional (σ, κ) metric, and the multi-agent extension.

11. Design Principles for AI Coding Agent Harnesses

The formal framework motivates six principles for harness design. We distinguish between principles that follow directly from the formal results (Principles 1, 3, 4) and those that are informed by the framework but draw primarily on empirical evidence or established software engineering wisdom (Principles 2, 5, 6):

1. **Operate at the Specification Level.** The harness should encourage human interaction at the specification layer ($\sigma > 0.5$), not the code layer. The human’s primary artifact should be a specification that the agent refines to code. This follows from Theorem 3.3: for complex programs, the specification *must* contain most of the program’s information regardless.
2. **Support Multiple Levels of Formalism.** Not every system component needs the same σ . A harness should support Gherkin/BDD for acceptance criteria ($\sigma \approx 0.4$), contracts for module interfaces ($\sigma \approx 0.7$), TLA+ for distributed protocols ($\sigma \approx 0.8$), and dependent types for critical invariants ($\sigma \approx 0.9$). AWS’s selective use of TLA+ for critical distributed protocols while using informal methods elsewhere validates this mixed-formalism approach (Newcombe et al.,

- 2015).
3. **Use AI as a Formalism Translator.** The historical barrier to formal methods was cost (Kleppmann, 2025). AI can serve as a translator: accepting NL intent and producing formal specifications for human review. Reviewing a specification is vastly easier than authoring one from scratch. This shifts the human role from *specification author* to *specification reviewer*.
 4. **Employ Automatic, Continuous Verification.** By Theorem 3.1(c), refinement steps reduce the abstraction gap (up to logarithmic terms). Proof checkers, model checkers, and property-based test generators should run on every change, providing deterministic verification that neutralises the stochasticity of LLM generation.
 5. **Select Coordination Topology by Task Structure.** As the empirical findings of Kim et al. (2025) show (Section 5), multi-agent topologies have distinct empirical profiles with respect to error amplification, coordination overhead, and output diversity. Centralised coordination reduces error amplification but adds overhead; independent agents maximise diversity but amplify errors. The harness should select topology based on task decomposability and required σ level.
 6. **Preserve the Theory.** Following Naur (1985), programs embody tacit theories that cannot be fully externalised. The harness should capture and curate the reasoning behind decisions: conversation logs, design rationale, specification evolution histories. This “theory preservation” addresses the information that formal specifications and code both fail to capture. We note that Naur’s position is stronger than “save metadata”: he argued that the theory held in programmers’ minds *cannot* be fully captured in any artifact. Our principle addresses what can be preserved, while acknowledging the residual gap.

11.1. A Testable Prediction

The framework generates at least one non-obvious, testable prediction. By Theorem 3.3, for sufficiently complex (incompressible) programs, the information content of a specification that uniquely determines the implementation must approach the information content of the implementation itself. This predicts a *crossover point*: for tasks below a certain complexity threshold, natural language specifications should suffice (because the program is compressible and the LLM’s prior can fill the gap); above that threshold, the pass rate of NL-specified agent runs should degrade sharply as the gap between the specification’s information content and the program’s complexity grows.

Concretely: if one measures task complexity by the normalised compression distance between minimal NL specification and correct implementation (an NCD proxy for $\mathcal{G}$), agent pass rates should exhibit a sigmoidal decline as NCD increases, with the inflection point corresponding to the regime where the LLM’s prior can no longer compensate for the information deficit. Existing benchmark data is suggestive (HumanEval at >90% for low-NCD tasks vs. LiveCodeBench at 38.9% for high-NCD tasks), but a controlled study varying specification detail at fixed task complexity would provide a direct test. We leave this experiment to future work.

12. Limitations

The framework has several important limitations. First, the central quantity $\mathcal{G}(S, P) = K(P \mid S)$ is uncomputable: no algorithm can compute the abstraction gap for an arbitrary specification-code pair. The specificity metric σ is likewise undecidable even for finite program spaces (by Rice’s theorem). The framework therefore serves as *conceptual scaffolding* that organises intuitions and connects disparate formalisms, rather than as a measurement tool. The Shannon channel model (Section 3) provides a computable, probabilistic counterpart, and we view the development of computable in-

stantiations with proved approximation bounds as the most important direction for future work.

Second, the design principles in Section 11 are motivated by, but not strictly derived from, the formal framework. We cannot currently identify a concrete case where the framework would have led to a design decision *different* from what experienced harness engineers already do. The framework’s value is in providing a unified vocabulary and theoretical grounding, not in overriding engineering judgment.

Third, the multi-agent analysis draws on empirical findings from Kim et al. (2025) without providing an original information-theoretic model of coordination overhead. Connecting the coordination metrics (O , A_e , R) to conditional Kolmogorov complexity remains an open problem.

13. Conclusion

We have presented a conceptual framework for the abstraction spectrum between human language and executable code. The central contribution is a novel application of algorithmic information theory to the specification-code relationship: the abstraction gap $\mathcal{G}(S, P) = K(P \mid S)$ provides a well-defined metric with proven monotonicity under refinement, and the convergence theorem (Theorem 3.3) formalises Gonzalez’s thesis that a sufficiently detailed spec must contain essentially all the information of the program. The Curry-Howard-Lambek correspondence establishes a structural isomorphism between specifications and code, while the refinement lattice gives the spectrum a rigorous partial order.

Analysis of multi-agent systems through this lens, drawing on empirical evidence from Kim et al. (2025), shows that coordination topology modulates error amplification by 4–17× depending on architecture. The two-dimensional specificity-compression metric reveals three clusters of thinkers and a hypothesis-generating “uncanny valley” of specification.

For harness designers, we offer six principles motivated by the framework: operate at the specification level, support multiple formalism levels, use AI as a formalism translator, employ continuous verification, select topology by task structure, and preserve the theory behind decisions. The vericoding pattern (human specification, AI implementation, formal verification) already achieves 82% on Dafny and represents the most promising near-term architecture.

The question is not whether humans should talk to machines in English or in code. It is where on a formally characterised spectrum the harness should place the human-machine interface. This framework provides a vocabulary and theoretical grounding for that question; computable, falsifiable instantiations remain the essential next step.

Acknowledgments

We thank the broader research community whose foundational work, spanning six decades from Dijkstra’s EWD 667 to modern AI coding agents, made this synthesis possible.

References

- J.-R. Abrial. *Modeling in Event-B: System and Software Engineering*. Cambridge University Press, 2010.
- S. Altman. Intelligence too cheap to meter. OpenAI Blog, 2024.

- R.-J. Back. *Correctness Preserving Program Refinements: Proof Theory and Applications*. Mathematisch Centrum, Amsterdam, 1980.
- F. P. Brooks. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4): 10–19, 1987.
- G. J. Chaitin. On the length of programs for computing finite binary sequences. *Journal of the ACM*, 13(4):547–569, 1966.
- F. Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- A. Clark. *Natural-Born Cyborgs: Minds, Technologies, and the Future of Human Intelligence*. Oxford University Press, 2003.
- C. Congdon. Formal methods in the age of AI, 2025.
- P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pages 238–252, 1977.
- E. W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.
- E. W. Dijkstra. On the foolishness of “natural language programming” (EWD 667). E.W. Dijkstra Archive, 1978.
- GitClear. AI copilot code quality research 2025. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2025.
- Gonzalez. A sufficiently detailed spec is code, 2026.
- T. R. G. Green and M. Petre. Usability analysis of visual programming environments: A “cognitive dimensions” framework. *Journal of Visual Languages & Computing*, 7(2):131–174, 1996.
- A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu. On the naturalness of software. *Communications of the ACM*, 59(5):122–131, 2012.
- C. A. R. Hoare. The emperor’s old clothes. *Communications of the ACM*, 24(2):75–83, 1981.
- C. A. R. Hoare and J. He. Unifying theories of programming. *Prentice Hall International Series in Computer Science*, 1998.
- E. Hutchins. *Cognition in the Wild*. MIT Press, 1995.
- K. E. Iverson. Notation as a tool of thought. *Communications of the ACM*, 23(8):444–465, 1980.
- D. Jackson. *The Essence of Software: Why Concepts Matter for Great Design*. Princeton University Press, 2021.
- J. Kim et al. Scaling LLM-based multi-agent systems for software engineering. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.
- M. Kleppmann. Formal methods: Just good engineering practice?, 2025.
- D. E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.
- A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.

- J. Lambek and P. J. Scott. *Introduction to Higher Order Categorical Logic*. Cambridge University Press, 1986.
- M. Li and P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 4th edition, 2019.
- M. Li, X. Chen, X. Li, B. Ma, and P. Vitányi. The similarity metric. *IEEE Transactions on Information Theory*, 50(12):3250–3264, 2004.
- G. Litt. Malleable software in the age of LLMs. <https://www.geoffreylitt.com/2023/03/25/llm-end-user-programming>, 2023.
- P. Martin-Löf. *Constructive Mathematics and Computer Programming*. North-Holland, 1979.
- METR. Do SWE-bench verified solutions actually get merged?, 2026.
- C. Morgan. *Programming from Specifications*. Prentice Hall, 2nd edition, 1994.
- P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985.
- C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.
- D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- C. E. Shannon. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1):50–64, 1951.
- L. Suchman. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987.
- R. S. Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019.
- VeriCoding. VeriCoding: AI-generated code with formal verification, 2025.
- P. Wadler. Propositions as types. *Communications of the ACM*, 58(12):75–84, 2015.
- L. Wittgenstein. *Philosophical Investigations*. Macmillan, 1953. Translated by G.E.M. Anscombe.

A. Full Proofs

A.1. Proof of Theorem 3.1: Properties of the Abstraction Gap

Proof. (a) Minimality. Suppose $\mathcal{G}(S, P) = K(P \mid S) = c$ for some constant c . Then there exists a program p with $|p| = c$ such that $U(p, S) = P$. Since c is a constant independent of $|S|$ and $|P|$, the specification S together with a fixed finite “translation key” p suffices to reconstruct P . We say S *algorithmically determines* P .

Conversely, suppose S algorithmically determines P . Then by definition there exists a fixed program p_0 (independent of $|P|$) such that $U(p_0, S) = P$, so $K(P \mid S) \leq |p_0| = O(1)$.

(b) Maximality. By the basic property of conditional complexity, $K(P \mid S) \leq K(P) + O(1)$ for all S (since we can always ignore the auxiliary input S and compute P from scratch). Equality $K(P \mid S) = K(P) + O(1)$ holds when $I(S : P) = K(S) + K(P) - K(S, P) = O(\log n)$, i.e., when the algorithmic mutual information between S and P is negligible. This occurs precisely when S provides no algorithmic information about P , i.e., the specification is informationally independent of (or irrelevant to) the program.

(c) Monotonicity. Suppose $S \sqsubseteq S'$ in the refinement ordering. By the chain rule for Kolmogorov complexity (which relies on the symmetry of information $K(x, y) = K(x) + K(y \mid x^*) + O(\log n)$; see [Li and Vitányi, 2019](#), Theorem 3.9.1):

$$K(P \mid S') \leq K(S \mid S') + K(P \mid S) + O(\log n)$$

General case (standard refinement). The refinement $S \sqsubseteq S'$ means every implementation of S' also implements S , but it does not guarantee that S is reconstructible from S' in bounded space (there may be many specifications that S' refines). In the worst case, $K(S \mid S') = O(\log n)$ (since S can be identified among $O(\text{poly}(n))$ candidates given S'), giving:

$$\mathcal{G}(S', P) \leq \mathcal{G}(S, P) + O(\log n)$$

Constructive refinement (stronger bound). If additionally there exists a computable function f of bounded description length such that $f(S') = S$, then $K(S \mid S') \leq |f| = O(1)$, and the bound tightens to:

$$\mathcal{G}(S', P) \leq \mathcal{G}(S, P) + O(1)$$

This stronger bound applies when the refinement step is witnessed by a concrete, short transformation, which is the case in most practical refinement calculus applications (e.g., data refinement with an explicit retrieve function).

(d) Additivity. This follows from the chain rule for Kolmogorov complexity. Given an intermediate representation S' :

$$\begin{aligned} K(P \mid S) &\leq K(S', P \mid S) + O(1) \\ &= K(S' \mid S) + K(P \mid S, S', K(S' \mid S)) + O(\log n) \\ &\leq K(S' \mid S) + K(P \mid S') + O(\log n) \\ &= \mathcal{G}(S, S') + \mathcal{G}(S', P) + O(\log n) \end{aligned} \tag{8}$$

The second-to-last inequality uses the fact that $K(P \mid S, S', K(S' \mid S)) \leq K(P \mid S') + O(1)$, since having S and $K(S' \mid S)$ in addition to S' cannot increase the complexity of producing P . The logarithmic term $O(\log n)$ arises from encoding the length $K(S' \mid S)$ itself, which requires $O(\log K(S' \mid S)) \leq O(\log n)$ bits. $\square$

A.2. Proof of Theorem 3.3: Spec-Code Convergence

Proof. Let P be an algorithmically random program, so $K(P) \geq |P| - O(1)$. Let S be a specification such that $P \models S$ and $\mathcal{G}(S, P) = K(P | S) = O(1)$.

Step 1. From $\mathcal{G}(S, P) = O(1)$, there exists a constant-length program p with $U(p, S) = P$. Therefore S together with p constitutes a description of P , giving:

$$K(P) \leq K(S) + |p| + O(\log(K(S) + |p|)) = K(S) + O(\log K(S)) \quad (9)$$

The logarithmic overhead arises from the need to delimit S within the concatenated description.

Step 2. Since $K(P) \geq |P| - O(1)$ (algorithmic randomness), we have:

$$|P| - O(1) \leq K(P) \leq K(S) + O(\log K(S)) \quad (10)$$

Step 3. Since $K(S) \leq |S| + O(1)$ (any string can be described by itself plus a constant overhead), we obtain:

$$|P| \leq |S| + O(\log |S|) + O(1) \quad (11)$$

To bound $\log |S|$: from Step 1, $K(S) \geq K(P) - O(1)$, and $K(P) \leq |P|$. Since $K(S) \leq |S| + O(1)$, we have $|S| \geq K(P) - O(1) \geq 0$, and trivially $|S| \leq |S| + |P|$, so $\log |S| = O(\log(|S| + |P|))$. But from Step 2, $|P| \leq K(S) + O(\log K(S)) \leq |S| + O(\log |S|)$, which gives $|S| + |P| = O(|S|)$, so $\log |S| \leq \log(|P| + O(\log |P|))$. More directly: from $K(S) \leq |P| + O(1)$ (combining Steps 1 and 2), we get $\log K(S) = O(\log |P|)$, and hence $\log |S| \leq \log(K(S) + O(1)) = O(\log |P|)$.

Rearranging: $|S| \geq |P| - O(\log |P|)$.

Interpretation. For large programs, $O(\log |P|)$ is negligible relative to $|P|$. Note that $|P|$ here denotes *string length in bits*, not lines of code. A 10,000-line program is approximately 4×10^6 bits (at ~ 50 characters/line, 8 bits/character), giving $O(\log 4 \times 10^6) \approx 22$ bits of slack, which is negligible. The specification must contain essentially all the information of the program; it has effectively become the program. $\square$

A.3. Derivation for Remark 5.1: Agent Decomposition

The decomposition follows directly from the chain rule. Let $\hat{S}$ denote the agent’s internal representation after processing specification S . By the chain rule for Kolmogorov complexity:

$$K(P | S) \leq K(\hat{S}, P | S) + O(1) = K(\hat{S} | S) + K(P | S, \hat{S}, K(\hat{S} | S)) + O(\log n) \quad (12)$$

Since $K(P | S, \hat{S}, K(\hat{S} | S)) \leq K(P | \hat{S}) + O(1)$ (extra information cannot increase complexity), we obtain:

$$\mathcal{G}(S, P) \leq \underbrace{K(\hat{S} | S)}_{\text{interpretation gap}} + \underbrace{K(P | \hat{S})}_{\text{generation gap}} + O(\log n) \quad (13)$$

The interpretation gap $K(\hat{S} | S)$ measures how much additional information the agent’s “understanding” contains beyond what was in the specification (e.g., training data priors, contextual inference). The generation gap $K(P | \hat{S})$ measures how much implementation detail remains after the agent has formed its internal representation. Note this holds for *any* intermediate string $\hat{S}$; the agent-specific content is entirely in the interpretation.

B. Extended Curry-Howard-Lambek Correspondence

The full three-way correspondence, extending the summary in Section 4:

Intuitionistic Logic	Type Theory (λ -calculus)	Category Theory (CCC)
Proposition P	Type A	Object A
Proof of P	Term $t : A$	Morphism $1 \rightarrow A$
Implication $P \Rightarrow Q$	Function type $A \rightarrow B$	Exponential B^A
Conjunction $P \wedge Q$	Product type $A \times B$	Product $A \times B$
Disjunction $P \vee Q$	Sum type $A + B$	Coproduct $A + B$
True ($\top$)	Unit type 1	Terminal object 1
False ($\perp$)	Empty type 0	Initial object 0
$\forall x:A. B(x)$	$\Pi(x:A).B(x)$	Right adjoint to pullback
$\exists x:A. B(x)$	$\Sigma(x:A).B(x)$	Left adjoint to pullback
Hypothetical $P \vdash Q$	Typing $\Gamma \vdash t : B$	Morphism $[\Gamma] \rightarrow [B]$
Cut elimination	β -reduction	Composition
Identity axiom	Identity function id_A	Identity morphism id_A
Modus ponens	Function application	Eval: $B^A \times A \rightarrow B$

Table 3 | The full Curry-Howard-Lambek correspondence. Each row is a formal isomorphism: the logical notion, the type-theoretic notion, and the categorical notion are three representations of a single mathematical structure. Source: [Lambek and Scott \(1986\)](#); [Wadler \(2015\)](#).

Significance for the abstraction spectrum. A specification expressed as a logical proposition (e.g., $\forall xs : \text{List } \mathbb{N}. \exists ys : \text{List } \mathbb{N}. \text{Sorted}(ys) \wedge \text{Perm}(xs, ys)$) is *simultaneously*:

- A **logical statement** (the specification),
- A **type** in dependent type theory (the signature of any correct sort function), and
- An **object** in a locally Cartesian closed category (whose global elements are the implementations).

Any program inhabiting this type is a *proof* of the specification. The “distance” from spec to code is the distance from knowing an object’s type to exhibiting a specific term, formalised by the refinement lattice (Section 4).

C. Empirical Calibration Data

C.1. Spec-to-Code Ratios from Real Projects

Project	Spec Size	Code Size	Ratio	Notes
Gonzalez YAML study	3,388 lines	16,063 lines	4.7:1	Semi-structured spec
seL4 (code/abstract spec)	7,000 lines Haskell	8,700 lines C	1.2:1	Abstract spec only
seL4 (full verification)	200,000+ lines Isabelle	8,700 lines C	0.04:1	Proof $\gg$ code
CompCert C compiler	28,000 lines Coq	6,000 lines C	0.2:1	Verified compiler
AWS TLA+ (DynamoDB)	~50 lines TLA+	~thousands	20–100:1	Per-component
Typical enterprise	~1 page NL	500–5,000 lines	500–5000:1	Highly variable

Table 4 | Spec-to-code ratios from real-world projects. The ratio varies by over three orders of magnitude depending on the formality of the specification. Data from [Gonzalez \(2026\)](#); [Newcombe et al. \(2015\)](#), seL4 project documentation, and CompCert project documentation.

C.2. Function Point Backfiring Ratios

Function points (FP) provide an empirical proxy for language abstraction level via the LOC/FP conversion ratio (Albrecht 1979; IFPUG/QSM):

Language	LOC/FP	α (abstraction)	σ (specificity)
Machine code	~640	0.00	1.00
Assembly	320	0.11	0.89
C	128–148	0.25	0.75
C++	53–59	0.37	0.63
Java	53–55	0.38	0.62
C#	54	0.38	0.62
Python	32	0.46	0.54
Ruby	25	0.50	0.50
Perl	21–27	0.53	0.47
SQL	12–13	0.60	0.40
Spreadsheet formulas	6	0.72	0.28
Natural language (est.)	~2	0.89	0.11

Table 5 | Abstraction and specificity levels derived from function point backfiring ratios. $\alpha(L) = 1 - \log(e(L))/\log(e_{\max})$ where $e(L) = \text{LOC}_L/\text{FP}$ and $e_{\max} = 640$ (machine code). $\sigma = 1 - \alpha$. The NL estimate assumes ~2 words per function point for a concise requirement. Source: QSM Function Point Languages Table; Capers Jones backfiring tables.

Key observation. The FP-based σ measures *language-level* specificity (how many lines of code a function point expands to), which correlates with but is distinct from *semantic* specificity (how many programs satisfy the spec). TLA+ is concise like NL ($\alpha \approx 0.85$) but semantically precise ($\sigma_{\text{semantic}} \approx 0.78$). This motivates the two-dimensional (σ, κ) metric in Section 6.

D. The Seven Answers to Gonzalez’s Question

Each intellectual tradition answers “Is a sufficiently detailed spec code?” differently. We include traditions that reject the question’s framing, since their objections illuminate assumptions that the information-theoretic framework takes for granted:

Thinker	Answer	Implication
Martin-Löf	Yes, by isomorphism	Types = specs = propositions. Programs = proofs. The Curry-Howard correspondence establishes structural identity, though not practical interchangeability.
Dijkstra	Yes, obviously	That is why we need formal notation. The desire for NL specification is the desire to be imprecise.
Brooks	Yes, and that’s the hard part	The specification IS the essential complexity. AI tools that eliminate “coding” eliminate only the easy part.
Parnas	Yes, but that’s a failure	A spec detailed enough to be code has collapsed information hiding. Good specs must be <i>less</i> detailed than code.
Kay	The question is wrong	Spec and code co-evolve through interactive exploration. The distinction dissolves in a live environment.
Kolmogorov	Yes, mathematically	For incompressible programs, $ S \geq P - O(\log P)$. The spec must contain the program’s information.
Rice	Yes, but you can’t verify it	Verification of conformance ($P \models S$) is undecidable in the general case. Convergence is real but undetectable.
Suchman	The question presupposes a false model	Specifications are not static plans but resources for situated action. The spec-code relationship is dynamic, not a fixed information-theoretic quantity.
Wittgenstein	The question’s meaning depends on its use	Whether a “detailed spec” counts as “code” is determined by the language game in which both terms are deployed, not by a formal metric.

Table 6 | How each tradition answers Gonzalez’s thesis. Formalists prove convergence as a theorem; pragmatists accept it as a design constraint; information theorists establish it as a mathematical necessity. Traditions rooted in situated cognition and ordinary language philosophy reject the question’s framing, challenging the assumption that specifications have fixed informational content.

E. Extended Thinker Placement: Key Quotes and Justifications

For each thinker in Table 1, the σ assignment is grounded in their primary works:

Dijkstra ($\sigma = 0.95$ – 1.0). “The virtue of formal texts is that their manipulations, in order to be legitimate, need to satisfy only a few simple rules” (EWD 667). “The naturalness [of natural language] boils down to the ease with which we can use them for making statements the nonsense of which is not obvious” (Dijkstra, 1978). Not exactly 1.0 because even Dijkstra acknowledged the need for informal reasoning at the problem-domain level.

Hoare ($\sigma = 0.85$ – 0.90). “There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies” (Hoare, 1981). The Hoare triple $\{P\}S\{Q\}$ specifies *what* without *how*, so multiple programs satisfy the same triple, hence $\sigma < 1.0$.

Knuth ($\sigma = 0.60$ – 0.65). “Let us change our traditional attitude to the construction of programs: Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do” (Knuth, 1984). Literate programs contain all the code *plus* NL explanation; the spec is interleaved with, not separate from,

the implementation.

Brooks (σ = variable). “I believe the hard part of building software to be the specification, design, and testing of this conceptual construct, not the labour of representing it” (Brooks, 1987). Brooks refuses a fixed σ because essential complexity varies by problem domain.

Parnas (σ = 0.50–0.60). “On the Criteria To Be Used in Decomposing Systems into Modules” (Parnas, 1972) established that specs *must* be less detailed than code. A Parnas-style tabular specification uses mathematical precision without algorithmic detail.

Lamport (σ = 0.75–0.80). “The problem is thinking programmatically ...the way to think outside of that box is to think mathematically” (Quanta Magazine, 2022). TLA+ specifications compress 20–100× relative to implementation (Newcombe et al., 2015) while remaining formally precise.

Karpathy (σ = 0.05–0.15). “The hottest new programming language is English” (X, January 2023). Represents the LLM-era extreme: NL, mediated by AI, is a sufficient specification medium. The σ is low because NL permits a vast and unpredictable set of possible implementations.

Chollet (σ = 0.3–0.7). Chollet’s Abstraction and Reasoning Corpus (ARC) (Chollet, 2019) operationalises abstraction as the ability to identify and apply novel patterns from minimal examples. His framework implies that the appropriate specification level depends on the *abstraction complexity* of the task: tasks requiring novel abstractions need higher σ (because the LLM’s prior cannot compensate), while tasks involving familiar patterns can succeed at lower σ . The wide range reflects this task-dependence. Unlike Brooks (who focuses on essential complexity), Chollet’s position is grounded in a concrete benchmark with measurable properties, making it more amenable to empirical calibration against the abstraction gap framework.

Kay (σ = 0.2–0.5). “The best way to predict the future is to invent it.” Kay’s Smalltalk and STEPS project (which aimed to express a complete personal computing environment in ~20,000 lines) demonstrate a concrete design philosophy: systems should be built through interactive exploration in a live environment where the boundary between specification and implementation dissolves. In Smalltalk’s live-object model, objects are simultaneously the specification (their protocol), the implementation (their methods), and the test environment (they can be inspected and modified at runtime). This places Kay in the middle ground rather than at an extreme: the specification *exists*, but it is not a static document; it is embedded in a running system. The κ estimate reflects the high compression achieved by STEPS-style architectures.

Suchman (σ : rejects the metric). “Plans are resources for action, not determinants of action” (Suchman, 1987). Suchman’s position is not simply that specifications are imprecise (low σ); it is that the entire concept of a fixed specificity level misrepresents the nature of human action. Specifications, in her account, are post-hoc rationalisations of situated behaviour, not inputs to a refinement process. We assign σ a ‡ marker rather than a numeric range because Suchman would reject the premise of the measurement. Her work remains in the table because the *objection itself* is informative: it marks the boundary of the framework’s applicability. The κ estimate (0.8–0.9) reflects that situated action descriptions are typically concise and informal.

Wittgenstein (σ : rejects the metric). “The meaning of a word is its use in the language” (Wittgenstein, 1953). Wittgenstein’s later philosophy implies that abstraction level is not intrinsic to a document but depends on the language game in which it is used. The same text can function as a high- σ spec in one context (a team with shared conventions that make every term precise) and a low- σ spec in another (a new team member who reads the same words with different assumptions). This is not a claim about vagueness; it is a claim about the nature of meaning itself. The framework’s presupposition that specifications have fixed informational content is precisely what Wittgenstein challenges.

Like Suchman, Wittgenstein receives a $\ddagger$ marker because his position resists placement on a numeric axis, and his inclusion marks a genuine limit of the information-theoretic approach.

F. Verification Roadmap

The theorems in this paper are stated informally but are in principle mechanisable. We outline a verification strategy using existing proof assistants:

Theorem	Proof Assistant	Existing Formalisation
Thm 3.1 (Gap Properties)	Coq	Forster (ITP 2022): Kolmogorov complexity in Coq
Thm 3.2 (NID Universality)	Coq	Partial: Li-Vitányi 2004 proof is constructive
Thm 3.3 (Convergence)	Coq	Follows from Chaitin’s theorem (Forster)
Thm 4.1 (Refinement Lattice)	Lean 4	mathlib: Order.CompleteLattice
Galois Connections	Lean 4	mathlib: GaloisConnection
Curry-Howard (Table 3)	Agda	Intrinsic: Agda is the correspondence

Table 7 | Verification roadmap. Lean 4 with mathlib provides the richest library for lattice theory and Galois connections. Coq (via Forster’s ITP 2022 formalisation) is the only proof assistant with mechanised Kolmogorov complexity. Agda’s dependent type system makes the Curry-Howard correspondence definitional rather than proven.

Formalisation targets (in priority order):

1. **Refinement lattice** (Lean 4): Define the specification type as a predicate transformer, prove the complete lattice structure using mathlib’s CompleteLattice typeclass. Entry point: Mathlib.Order.CompleteLattice.
2. **Galois connections** (Lean 4): Formalise the tower of abstractions using mathlib’s GaloisConnection structure. Prove monotonicity of information loss. Entry point: Mathlib.Order.GaloisConnection.
3. **Gap properties** (Coq): Extend Forster’s Kolmogorov complexity formalisation with conditional complexity $K(x \mid y)$ and prove Theorem 3.1. This requires formalising the chain rule.
4. **Convergence theorem** (Coq): Prove Theorem 3.3 from the formalised Chaitin incompleteness bound.
5. **NID metric properties** (Coq/Lean 4): Prove the metric space axioms for $d(S, P)$. The triangle inequality proof requires careful handling of the $O(\log n/n)$ error terms.

Full mechanised verification of all theorems is estimated at 2,000–5,000 lines of proof code, achievable in 2–4 person-months for a researcher experienced with the respective proof assistants.

